

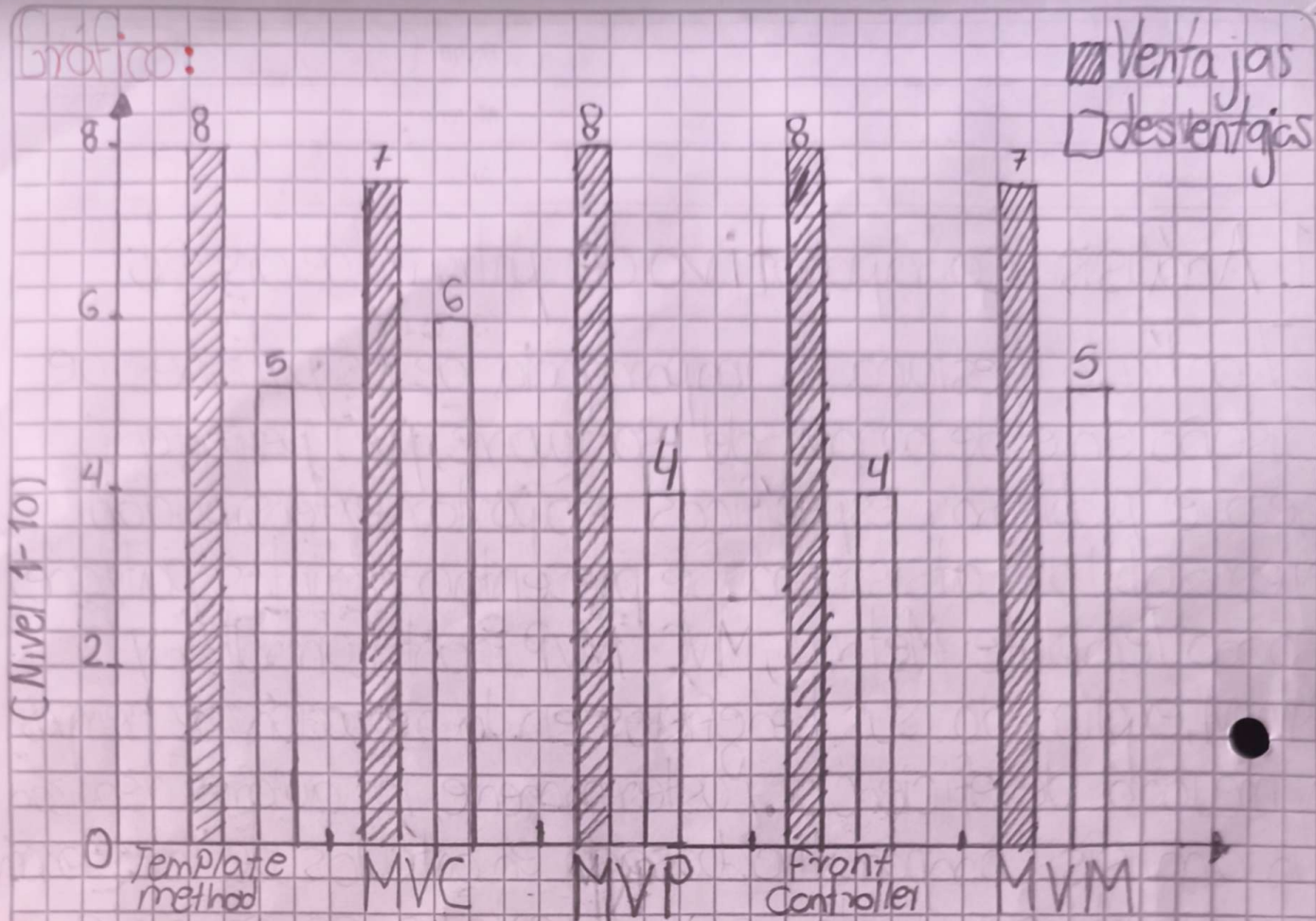
1. Análisis Comparativo de patrones de diseño

El artículo destaca la importancia de los patrones de diseño en el desarrollo de software, pues permiten resolver problemas específicos y aportar extensibilidad y escalabilidad al sistema. Se presentan distintos patrones como *Template Method*, *MVC*, *MVP*, *Front Controller* y *MVVM*, explicando sus beneficios en la reducción de tiempos y mejora de eficiencia. Posteriormente, los autores realizan un análisis comparativo basado en artículos de investigación para identificar ventajas y desventajas de cada uno.

Finalmente, concluyen que la elección del patrón depende del tipo de proyecto, la experiencia del equipo y el equilibrio entre mantenibilidad, rendimiento y complejidad.

Además de esto, los patrones de diseño son considerados una herramienta fundamental en la ingeniería de software moderna, los autores resaltan que conocer y dominar diversos patrones proporciona a los desarrolladores una base sólida de buenas prácticas que favorece la colaboración y el entendimiento común dentro del equipo de trabajo.

Gráfico:



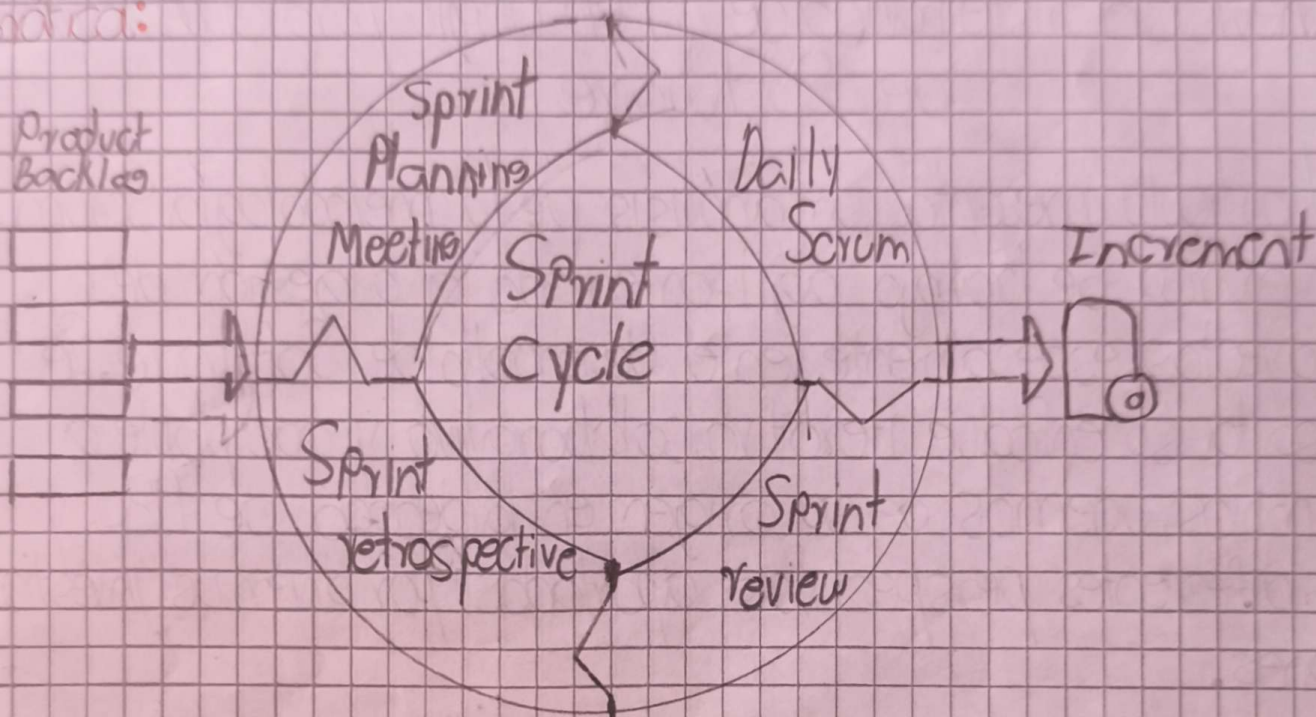
Reflexión: Mi reflexión personal sobre este tema es que los patrones de diseño representan una base indispensable en el desarrollo de software, ya que permiten que los proyectos no solo sean más organizados, sino también más fáciles de mantener a largo plazo. Considero que dominar estos patrones es una manera de pensar estratégicamente, pues cada patrón brinda soluciones probadas que evitan la reinención de la rueda y reducen errores comunes. Al mismo tiempo, pienso que no se trata de aplicar un patrón por obligación, sino de analizar el contexto y las necesidades del proyecto para elegir el más adecuado buscando siempre un equilibrio entre simplicidad y eficiencia. En definitiva, considero que son una herramienta poderosa que todo desarrollador debe conocer para ser profesionalmente.

2. Revisión Sistemática de la Metodología para el Desarrollo de Software

El artículo presenta un análisis de la metodología Scrum, un marco de trabajo ágil orientado a la gestión de proyectos, especialmente en el desarrollo de Software. Se resalta su enfoque iterativo, colaborativo y adaptable a cambios, además de su origen en la década de 1980 y las diferentes perspectivas aportadas por diversas investigaciones.

Los autores aplican una revisión sistemática basada en artículos y bases de datos especializadas para evaluar su aplicación en este ámbito. Concluyen que Scrum proporciona un entorno flexible y controlado, fomenta el trabajo en equipo y facilita la resolución conjunta de problemas. No obstante, advierten que resulta más adecuado en proyectos con menor volumen de información y que su éxito depende de la correcta conformación de los equipos, evitando interferencias que comprometan la integridad del proyecto.

Infografía:



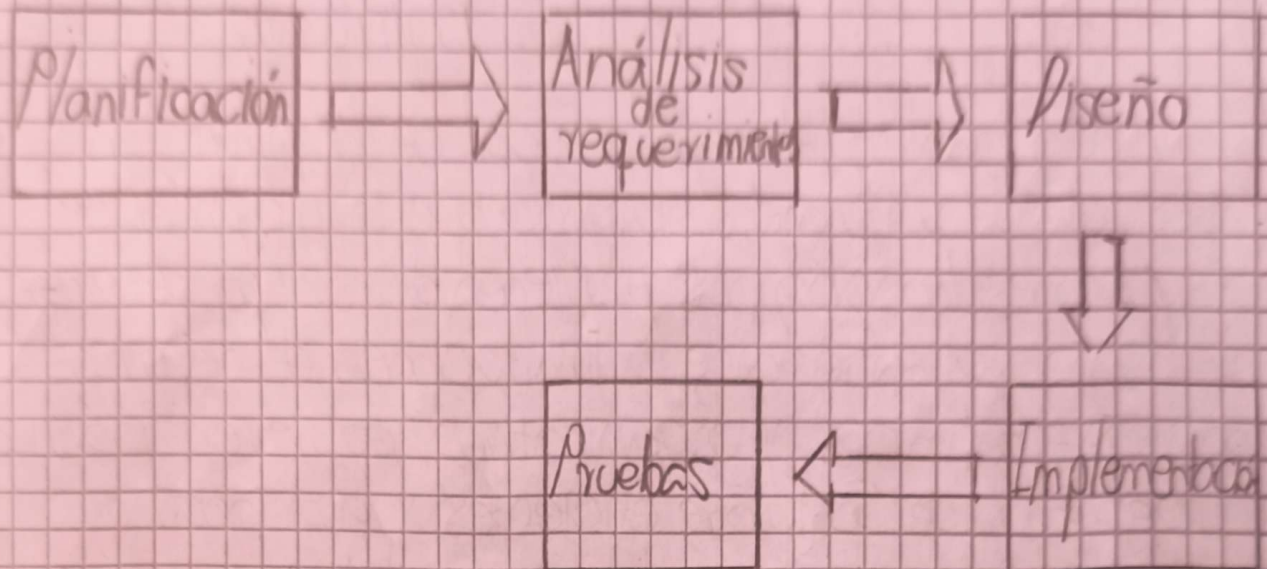
Reflexión: La lectura sobre la metodología Scrum me dejó una reflexión interesante acerca de cómo la gestión de proyectos en el ámbito del software ha evolucionado hacia un enfoque más humano, colaborativo y flexible. Scrum demuestra que no basta con tener un plan rígido, sino que es necesario contar con un equipo capaz de adaptarse a los cambios y responder con rapidez a las necesidades del entorno. Me parece valioso que se destaque la importancia del trabajo en equipo y la comunicación, pues muchas veces los proyectos fracasan no por falta de conocimientos técnicos, sino por problemas de coordinación y organización. También considero relevante la advertencia sobre sus limitaciones: no todos los proyectos son aptos para este marco de trabajo y su éxito depende en gran medida de la disciplina y compromiso de los integrantes.

3. Perfiles del ciclo de vida del Software para pequeñas empresas: los informes técnicos ISO/IEC 29110

El artículo analiza el papel crucial de la pymes en el desarrollo de software, destacando que la mayoría de las soluciones tecnológicas provienen de este tipo de empresa. Sin embargo, al enfrentarse a limitaciones de recursos humanos, económicos y organizativos, requieren prácticas de Ingeniería de Software eficientes y adaptadas a su contexto. Para entender esta necesidad, la ISO creó el grupo de trabajo SC7-WG24, cuyo objetivo es elaborar un estándar internacional que facilite la mejora de procesos de software específicamente en pymes. Este grupo busca establecer un marco común de perfiles evaluables del ciclo de vida de software, basados en la norma ISO/IEC 29110, lo que permitirá a estas empresas implementar procesos de calidad sin los altos costos de modelos tradicionales como CMMI o ISO 9001.

Al final del artículo se resalta que la iniciativa de ISO representa una oportunidad para que las pymes alcancen mejores niveles de productividad y competitividad en un entorno globalizado y cambiante. No obstante, también se reconocen las barreras que han dificultado la adopción de modelos más complejos en este tipo de organizaciones.

Gráfico:

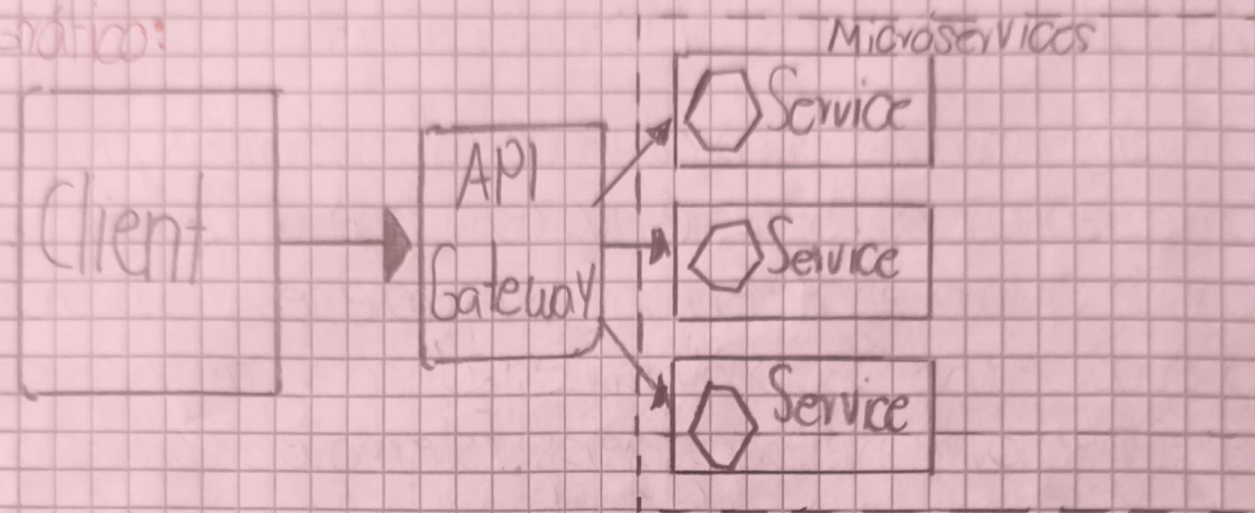


Reflexión: La lectura trata de la importancia de las pymes en el desarrollo de software permite reflexionar sobre cómo estas organizaciones, a pesar de sus limitaciones de recursos, tiene un papel fundamental en la innovación tecnológica. Su capacidad de adaptación y creatividad les otorga un valor estratégico dentro de la industria, pero al mismo tiempo plantea la necesidad de contar con metodologías y estándares que se ajusten a su realidad. La creación de la norma **ISO/IEC 20110** representa un paso significativo ya que reconoce las particularidades de las pymes y brinda herramientas prácticas para mejorar la calidad de sus procesos sin incurrir en los elevados costos. Esto refleja la importancia de diseñar marcos de trabajo inclusivos, accesibles y flexibles, que permitan a las pymes competir de manera más justa en un mercado global.

4. Arquitectura de Microservicios

El artículo introduce el concepto de microservicios, un enfoque de diseño de Software que consiste en dividir una aplicación en pequeños servicios independientes, cada uno encargado de una funcionalidad específica y comunicándose entre sí mediante interfaces ligeras. Este modelo contrasta con las arquitecturas monolíticas tradicionales, ya que busca mejorar la escalabilidad, flexibilidad y mantenibilidad de las aplicaciones, respondiendo a las crecientes demandas de los sistemas modernos. El autor plantea un ejemplo comparativo para ilustrar la diferencia entre ambas arquitecturas. En un sistema monolítico, a medida que la aplicación crece, el código se vuelve más complejo y difícil de mantener. En cambio, con una arquitectura de microservicios, cada módulo o servicio funciona de manera independiente. Además, se destaca que los microservicios facilitan la integración de nuevas tecnologías, permiten una mayor especialización de equipos de trabajo y ofrecen la posibilidad de desplegar y actualizar funcionalidades sin afectar el resto del sistema. Finalmente el autor invita a reflexionar sobre cómo esta arquitectura brinda soluciones adaptadas a las necesidades actuales del desarrollo de software, dividiendo un problema complejo en subsistemas más pequeños e independientes.

Gráfico:

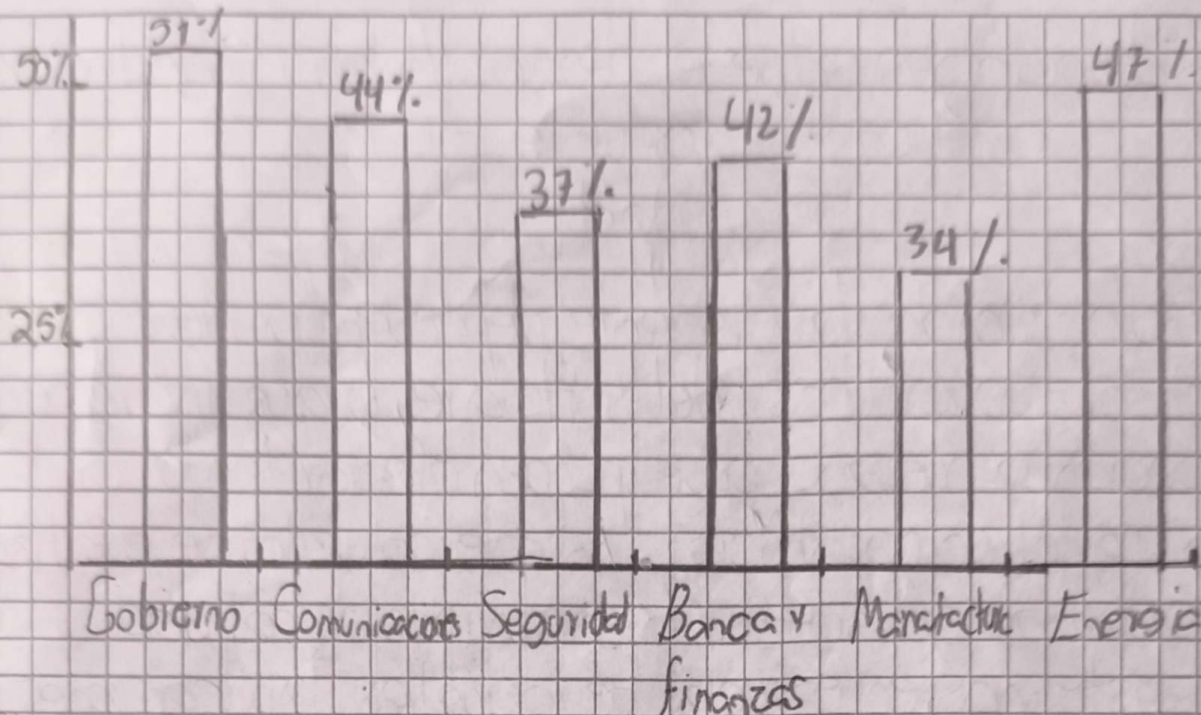


Reflexión: La arquitectura de microservicios representa una evolución significativa en la manera de concebir el desarrollo de software, especialmente frente a los retos actuales de la escalabilidad, flexibilidad y rápida adaptación a los cambios. A diferencia de los sistemas monolíticos, que suelen convertirse en estructuras rígidas y difíciles de mantener, los microservicios apuestan por la independencia de cada módulo, lo que genera mayor orden y control en el crecimiento de las aplicaciones. Esta independencia no solo facilita la incorporación de nuevas tecnologías y actualizaciones sin afectar la totalidad del sistema, sino que también fomenta la especialización y autonomía de los equipos de desarrollo. Sin embargo, también plantea desafíos, como la necesidad de una adecuada coordinación entre servicios y el manejo de la complejidad en la comunicación entre ellos. Los microservicios se consolidan como un enfoque moderno que ofrece soluciones efectivas para la construcción de sistemas robustos, escalables y sostenibles en el tiempo.

5. Consideraciones sobre el desarrollo en infraestructuras críticas y software de seguridad

El artículo aborda la creciente dependencia de las infraestructuras críticas del software, lo que incrementa la exposición a riesgos que pueden afectar tanto al mundo digital como al físico. En este contexto, se enfatiza la necesidad de garantizar la seguridad y calidad de los sistemas, en especial de aquellas aplicaciones consideradas críticas para el funcionamiento de una organización o sector. Se analiza el papel de los sistemas de Gestión de Seguridad de la información (CSGSI), regulados por las normas ISO 27001, como herramientas clave para reducir amenazas y asegurar la continuidad operativa. Diversos estudios citados coinciden en que el mayor riesgo de seguridad no proviene de agentes externos, sino de errores internos, malas prácticas adoptadas dentro de las propias organizaciones.

Finalmente, el artículo concluye que el gran reto consiste en garantizar que el desarrollo de Software crítico cumpla con altos estándares de calidad y seguridad, asegurado su propia resiliencia de los sistemas ante posibles amenazas.



Porcentaje de organizaciones que implementan medidas de eliminar o destruir información por tipo

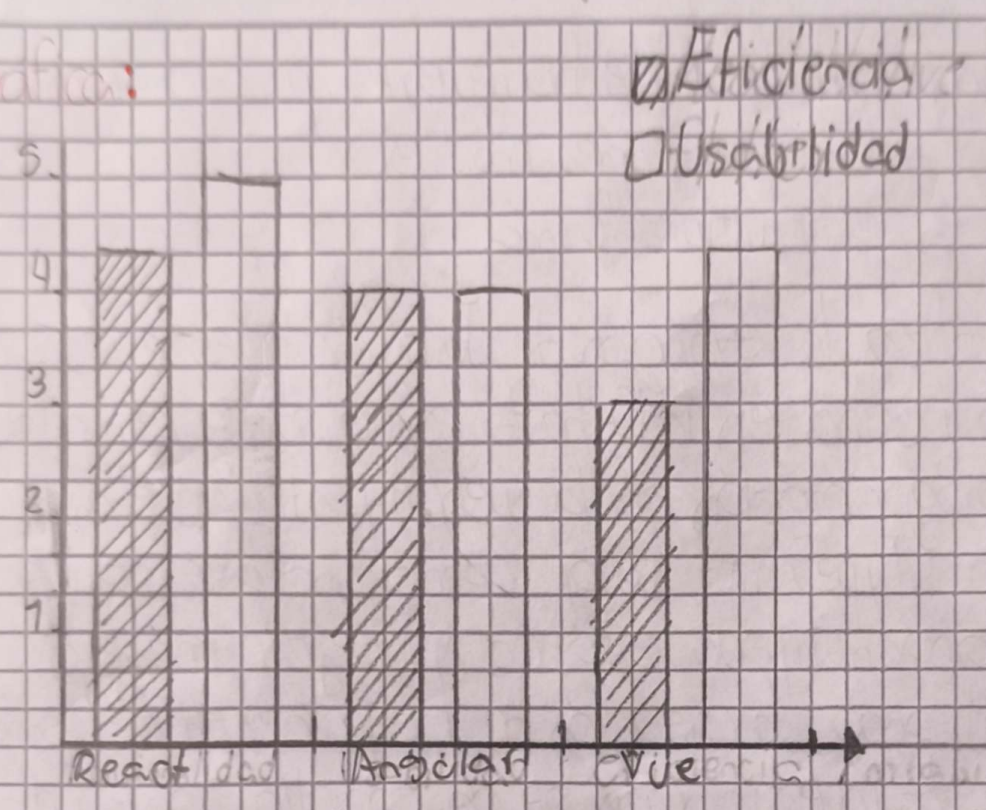
Reflexión: La lectura me deja claro que la seguridad en el desarrollo de software crítico no depende únicamente de las herramientas o normas implementadas, sino de la cultura organizacional y de la responsabilidad de quienes participan en el proceso. Muchas veces se piensa que las amenazas provienen solo del exterior, pero los errores humanos y las malas prácticas internas son un riesgo igual o incluso mayor. Esto refleja la necesidad de fomentar la capacitación constante, la disciplina en los procesos y la adopción de estándares como ISO 27001 no solo como requisito técnico, sino como un compromiso integral de la organización.

6. Análisis comparativo entre frameworks, para el desarrollo de aplicaciones móviles multiplataforma

Esta tesis analiza la evolución de JavaScript desde sus inicios como Mocha en 1995 hasta consolidarse como la base de la programación en la web. A partir de ello, se resalta cómo TypeScript surge como una alternativa robusta al añadir tipado estático y mejor manejo de datos, facilitando la creación de proyectos más seguros y escalables. La investigación se desarrolló con estudiantes de octavo semestre, durante un periodo de 6 semanas, empleando un enfoque práctico y progresivo. Se abordaron actividades que incluyeron la creación de servidores, el consumo de información en formato JSON y la aplicación de arquitecturas como MVC. En este contexto, se analizaron tres de los principales frameworks modernos: Angular, React y Vue. Los resultados indicaron que el framework Angular, a pesar de su curva de aprendizaje más extensa, proporciona una base sólida que permite comprender otros marcos con mayor facilidad. React destacó por su sencillez y compatibilidad directa con TypeScript y HTML, mientras que Vue resultó más intuitiva, especialmente después de experiencias previas en programación.

El estudio concluye que los frameworks en TypeScript impulsan la productividad, calidad del código y preparación profesional, ofreciendo metodologías centradas en la práctica y la resolución de problemas.

Gráfica:



Reflexión: La evolución de JavaScript y la incorporación de TypeScript evidencian cómo la programación web se ha fortalecido con herramientas que no solo amplían las posibilidades técnicas, sino que también genera un impacto formativo. La experiencia con estudiantes de Ingeniería en Sistemas refleja que el aprendizaje práctico, vinculado con frameworks modernos, no solo enriquece el conocimiento técnico, sino que también aproxima a los jóvenes al entorno laboral. Además, la comparación entre Angular, React y Vue demuestra que cada tecnología tiene fortalezas específicas que se ajustan a distintos perfiles y necesidades, lo que resalta la importancia de la adaptabilidad y el aprendizaje continuo.

Formulario de registro:

- Nota:
- Curso:
- Institución:
- Materia:
- Profesor:

7. Evaluación del rendimiento de los motores de bases de datos MySQL y Firebird

El artículo presenta una investigación orientada a evaluar el rendimiento de dos sistemas de gestión de bases de datos ampliamente utilizados: MySQL y Firebird, para ello, se diseñó e implementó una aplicación genérica de inventarios que permitió comparar el comportamiento de ambos motores en escenarios controlados. El proceso metodológico incluyó la creación de un algoritmo denominado *Robot*, cuya función principal fue poblar automáticamente las bases de datos con información generada de manera pseudorandom. Posteriormente, se establecieron casos de prueba que facilitaron la ejecución de transacciones y la medición de tiempos de respuesta. Los resultados mostraron que MySQL presentó mayor rapidez ejecutando las operaciones en menor tiempo que Firebird. No obstante, los autores señalan que ofrecer un juicio absoluto sobre qué motor resulta inviable, ya que cada sistema posee fortalezas y debilidades. Finalmente, se concluye que realizar este tipo de estudios con software libre aporta ventajas al ser accesible para instituciones y desarrolladores, contribuyendo tanto al aprendizaje como la optimización de recursos en proyectos reales.



Reflexión: La comparación entre MySQL y Firebird evidencia que en el ámbito de las bases de datos no existe una solución universal, sino herramientas que deben seleccionarse según las necesidades de cada proyecto. El hecho de que MySQL obtuviera un mejor desempeño en los experimentos no significa que Firebird carezca de valor, ya que este último puede resaltar en escenarios específicos dependiendo de factores como la seguridad, la escalabilidad o el tipo de aplicación. Lo más relevante del estudio no es únicamente el resultado técnico, sino la metodología aplicada. Esto resalta la importancia de evaluar con criterios objetivos antes de tomar decisiones tecnológicas.

8. Frameworks basados en TypeScript para el desarrollo de aplicaciones web interactivas

Este artículo busca entender la evolución de las tecnologías web y la relevancia de **TypeScript** como lenguaje que amplía las capacidades de JavaScript en el desarrollo moderno. Se destacan los frameworks **Angular**, **React** y **Vue** como herramientas fundamentales para la creación de aplicaciones interactivas, dinámicas y con bajo tiempo de respuesta. Además se resalta la importancia de **WebSockets** en la comunicación entre cliente y servidor, ya que permiten conexiones persistentes y de baja latencia, esenciales para aplicaciones en tiempo real. Los resultados obtenidos en la implementación con estudiantes reflejan que **Angular** posee una curva de aprendizaje más compleja, aunque una vez dominado facilita la transición hacia otros (frameworks) frameworks.

De igual forma, se concluye que el uso de técnicas de normalización, así como de hojas de estilos **CSS**, **SASS** y **LESS**, contribuye a mejorar la estructura, calidad y escalabilidad del código.

En conclusión, la experiencia con frameworks basados en TypeScript favorece el desarrollo de competencias técnicas, incrementa la productividad y prepara a los estudiantes para integrarse en entornos profesionales de desarrollo web.

Gráficos:



React.js

Vs



Angular

Vs



Vue

Reflexión: El avance de las tecnologías web no solo transforma la manera en que se crean aplicaciones, sino también la forma en que los desarrolladores adquieren y fortalecen sus competencias. La incorporación de nuevas herramientas implica enfrentarse a retos iniciales de aprendizaje que, aunque parezcan complejos, se convierten en oportunidades para desarrollar habilidades de adaptación y resolución de problemas. Este proceso de formación técnica no se limita únicamente al dominio de un lenguaje o entorno específico, sino que prepara a los profesionales para afrontar con mayor seguridad los cambios constantes que caracterizan a la industria de software. Asimismo, la práctica con metodologías modernas y el uso de técnicas que promuevan la calidad de código.

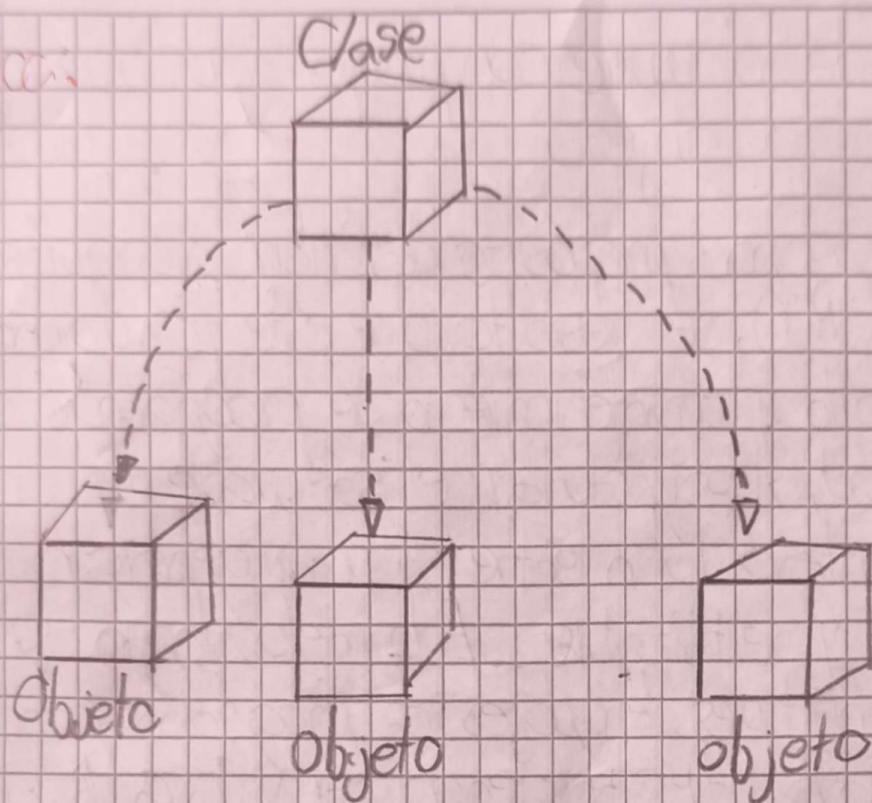
Nombre:		Institución:	
Materia:		Curso:	
Fecha:		Nota:	

9. Arquitectura de Software con programación orientada a objeto

El artículo tiene como propósito describir la arquitectura de Software con POO, se resalta que este paradigma facilita el desarrollo y mantenimiento de programas de gran complejidad, al estructurar el software en clases y objetos que interactúan entre sí para generar sistemas más funcionales y reutilizables. Esta metodología se ha incorporado en múltiples lenguajes de programación, tales como Python, Java, PHP, C++, siendo C++ y Java los más utilizados en la actualidad. El enfoque orientado a objetos promueve la reutilización de código, la modularidad y la escalabilidad, características esenciales para enfrentar los retos de la ingeniería de software moderna. Asimismo, se evidencia cómo la evolución de la arquitectura de software, ha permitido a los profesionales del área abordar problemas complejos de forma más eficiente.

La programación orientada a objetos aplicada a la arquitectura de Software no solo ofrece un marco sólido para el diseño y construcción de sistemas, sino que también impulsa la innovación y la mejora continua en el desarrollo de Software.

Gráfica:



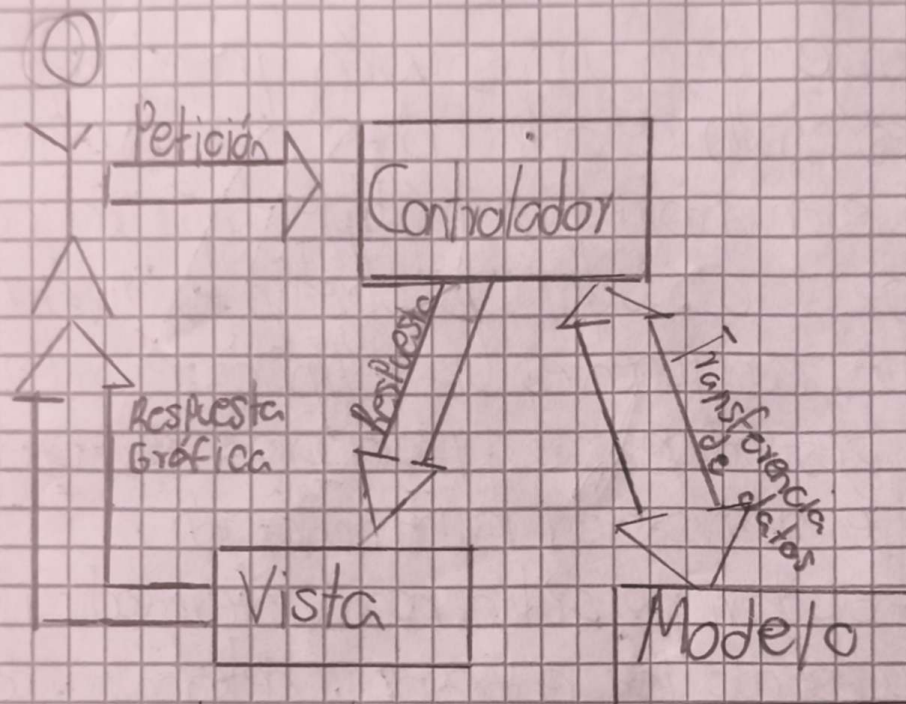
Reflexión: La programación orientada a objetos aplicada a la arquitectura de software representa un avance significativo en la forma en que se diseñan y mantienen los sistemas actuales. Su enfoque modular y reutilizable no solo facilita el trabajo de los desarrolladores, sino que también garantiza que los proyectos puedan actualizarse y adaptarse a nuevas necesidades con mayor facilidad. Este paradigma no es únicamente una técnica de programación, sino un pilar que fortalece la innovación y la evolución tecnológica ya que permite transformar ideas complejas en soluciones claras y sostenibles. En un mundo donde el software es cada vez más esencial en todos los ámbitos.

10. Patrón Modelo-Vista-Controlador

El patrón MVC surge como una solución para simplificar y optimizar el desarrollo de aplicaciones, especialmente aquellas que requieren manejar múltiples representaciones y sincronización de datos. Su principal objetivo es dividir el Software en tres componentes bien definidos: el modelo, encargado de la gestión de datos y la lógica de negocio; la Vista, responsable de mostrar la información al usuario; y el controlador, que actúa como intermediario entre ambos, procesando las entradas y determinando las respuestas adecuadas. Esta separación estructural permite que cada componente pueda implementarse y modificarse de manera independiente, lo cual facilita la actualización, mantenimiento y escalabilidad de las aplicaciones. Gracias a ello el código resulta más comprensible, ordenado y predecible, evitando la mezcla de funcionalidades que generan errores y dificultades en el desarrollo.

Esto contribuye no solo a mejorar la productividad, sino también a garantizar una mayor calidad en el producto final. En síntesis, el patrón MVC representa un paradigma esencial para la ingeniería de Software moderna, ofreciendo eficiencia, claridad y robustez en el desarrollo de sistemas.

Gráfica:



Reflexión: La reflexión sobre el patrón **MVC** destaca que su verdadero valor no solo radica en la organización del código, sino en cómo transforma la forma de pensar el desarrollo de software. Al separar responsabilidades, los desarrolladores pueden enfocarse en resolver problemas específicos sin generar dependencias innecesarias, lo que reduce la complejidad y los riesgos de errores. Además **MVC** fomenta el trabajo colaborativo, ya que distintos equipos pueden trabajar en paralelo sobre el modelo la vista o el controlador sin interferir entre sí. Esto no solo incrementa la productividad, sino que también impulsa la innovación.

Nota:

Curso:

Institución:

Materia:

Profesor:

41. Análisis del proceso de pruebas de calidad de Software

El artículo aborda la importancia del proceso de pruebas de calidad de Software como un componente estratégico dentro del ciclo de vida del desarrollo. A partir de la revisión de literatura científica de bases de datos reconocidas, así como de la experiencia del autor en la industria y la docencia, se analizan conceptos, metodologías y marcos de referencia que permiten comprender el valor de las pruebas en la construcción de productos confiables. La investigación evidencia que un proceso adecuado de pruebas contribuye a optimizar tiempos, reducir riesgos, mejorar la efectividad y aumentar la satisfacción de los usuarios finales.

Se destaca del artículo en diseñar e implementar un proceso formal de pruebas es esencial para garantizar estándares de calidad y fiabilidad en los productos, además fomenta el control de defectos y fallos en todas las etapas del proyecto.

Nombre:

Fecha:

Profesor:

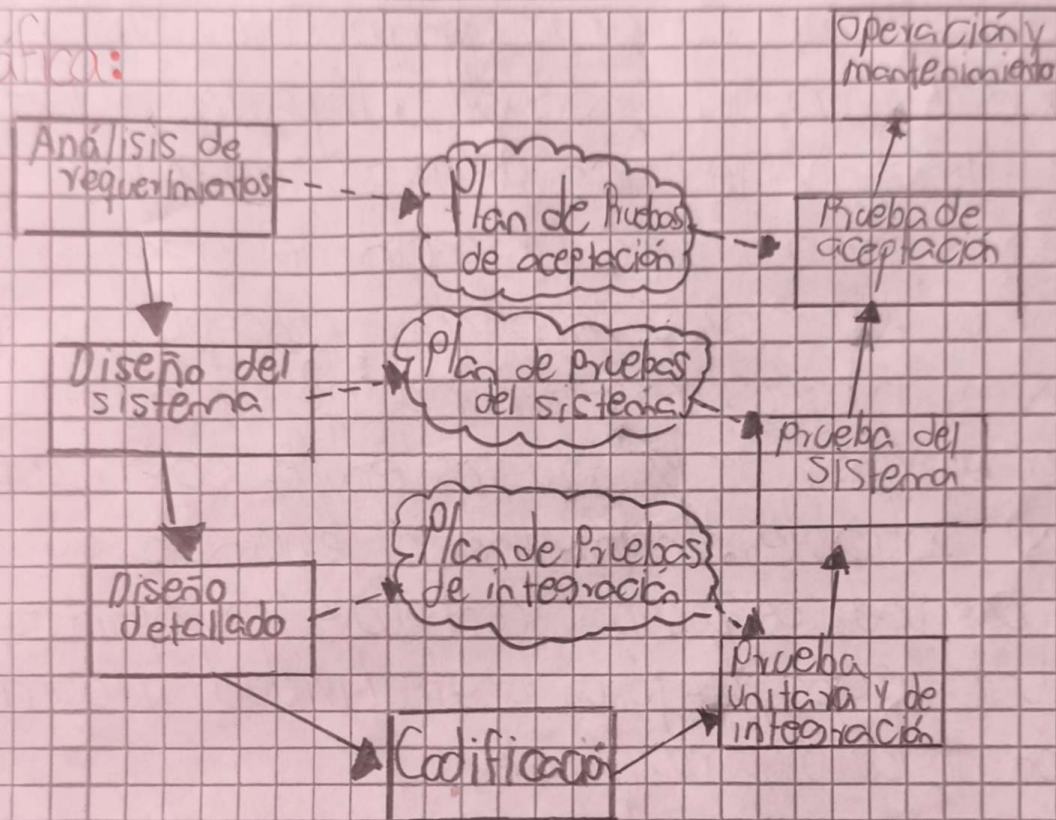
Materia:

Institución:

Curso:

Nota:

Gráfica:



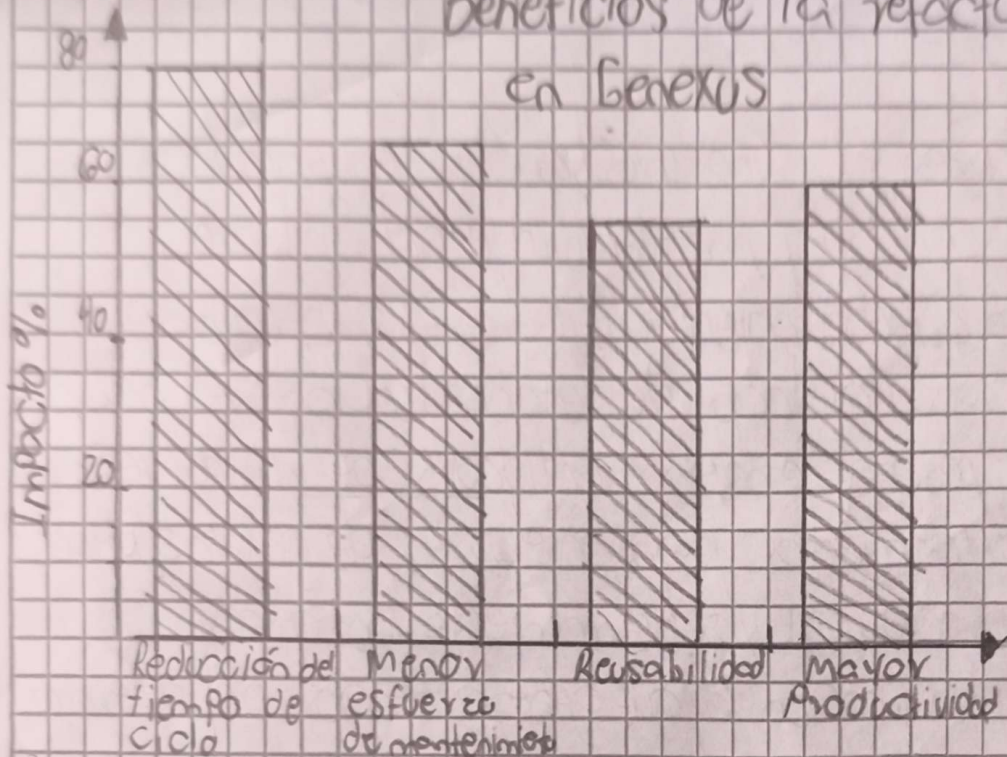
Reflexión: La lectura de este artículo permite reflexionar sobre cómo las pruebas de calidad de software no son simplemente una etapa técnica, sino un componente estratégico que define la confiabilidad y sostenibilidad de un producto en el mercado. Un sistema puede tener un diseño atractivo o funcionalidades avanzadas, pero si carece de un proceso formal de pruebas, su valor se ve comprometido por la posibilidad de fallos y defectos que afecten directamente al usuario final. Esto resalta la importancia de integrar las pruebas como parte del ciclo de vida del desarrollo y no como un complemento secundario. Invertir en la formación en pruebas de software no solo mejora la calidad de proyectos, sino que también impulsa la competitividad empresarial y la confianza de los clientes.

12 Catálogo de refactorización de código fuente para herramienta CASE Genexus

Este artículo presenta una investigación orientada a mejorar la calidad y mantenibilidad del Software desarrollado con la herramienta **Case Genexus** mediante la creación de un catálogo de refactorización de código fuente. Genexus permite modelar entidades del mundo real y definir comportamientos, pero en el caso de objetos basados en la lógica procedural, la herramienta no ofrece consistencia ni optimización lo que genera riesgos de deterioro del software con el tiempo. Para enfrentar este problema se estableció un conjunto de reglas de refactorización que permiten optimizar las aplicaciones, mantener su congruencia y facilitar la comprensión del código. Los resultados mostraron que la refactorización reduce significativamente el tiempo de ciclo en tareas de mantenimiento evidenciando mayor uniformidad y menor esfuerzo en comparación con proyectos sin refactorización donde los tiempos eran variables e inconsistentes.

El artículo propuesto contribuye en explicar directamente la importancia de **Case Genexus** en la calidad del software, ya que promueve la inteligibilidad, reusabilidad y reducción de defectos.

Beneficios de la refactorización en Genexus



Reflexión: La investigación resalta como la refactorización no solo es una técnica, sino una estrategia clave para garantizar la sostenibilidad de los proyectos de software en el tiempo. En el caso de Genexus, donde existen limitaciones para optimizar automáticamente el código el catálogo propuesto demuestra que aplicar reglas claras y sistemáticas aporta beneficios tangibles tanto en la mantenibilidad como en la calidad del producto final.

Este aporte es particularmente valioso porque evidencia que la estandarización del código facilita el trabajo en equipo, disminuye la dependencia de desarrolladores específicos y hace que nuevos integrantes se adapten más rápido a los proyectos. Asimismo, la reducción de tiempos y costos de mantenimiento.

Institución:	Curso:	Nota:
Profesor:	Materia:	
Nombre:	Fecha:	

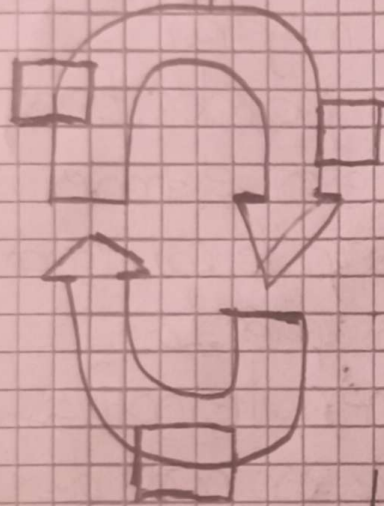
13. Modelos de Desarrollo de Software

El desarrollo de software ha enfrentado históricamente múltiples problemas de calidad, costos y tiempos, lo que dio origen al concepto de la "crisis del software" en 1968. Desde entonces, la ingeniería de Software ha buscado metodologías y modelos que permitan mejorar la eficiencia, organización y resultados de los proyectos. Estos modelos constituyen guías que abarcan fases esenciales como la especificación, el análisis, el diseño, la codificación, las pruebas y la implementación en entornos reales. La investigación tiene como objetivo proponer alternativas viables para la selección de un modelo de desarrollo de software, tomando en cuenta las características de cada proyecto y las particularidades de los equipos de trabajo. Para ello, se realizó un estudio comparativo entre diferentes modelos, lo que permitió identificar criterios de selección basados en la efectividad, la calidad de los resultados, el orden en la documentación y la capacidad de cumplir con los plazos establecidos. Los resultados obtenidos demuestran que la elección adecuada del modelo de desarrollo incide directamente en la calidad final del software, la organización del proceso y la satisfacción del usuario.

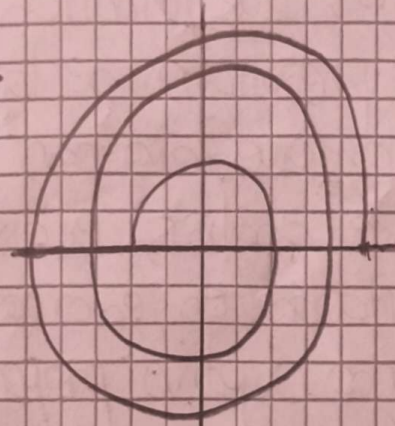
Nombre: Fecha: Profesor: Materia: Institución: Curso: Nota:

Gráfica:

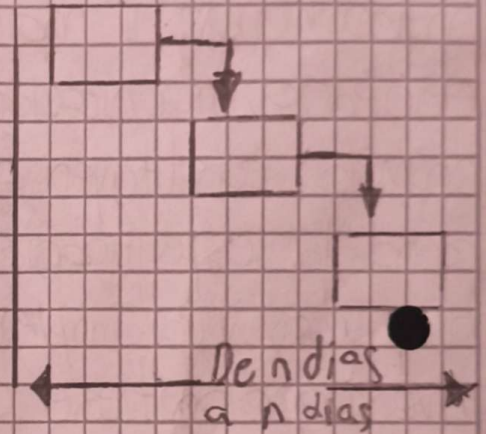
1. Prototipo:



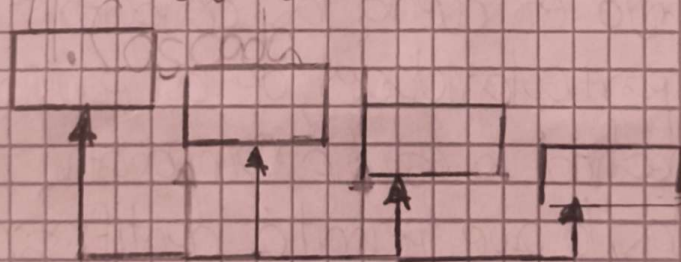
2. Espiral:



3. RAD:



4. Cascada:



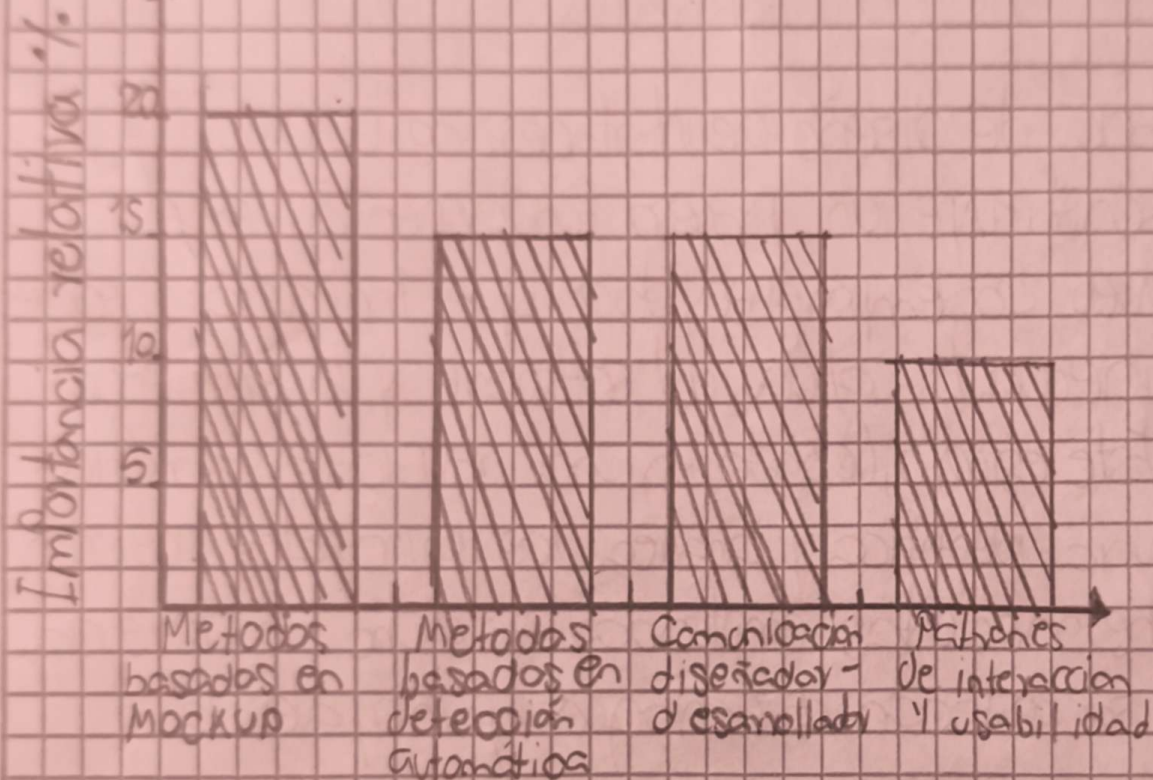
Reflexión: La investigación evidencia que la elección del modelo de desarrollo no debe ser un proceso arbitrario, sino una decisión estratégica que impacta de manera directa en la calidad del producto, la organización del trabajo y la satisfacción del cliente. La llamada "crisis del software" fue un punto de inflexión que permitió reconocer la necesidad de metodologías estructuradas y este estudio demuestra que aún hoy, la correcta selección de un modelo ideal, sino que la efectividad depende de factores como el tipo de proyecto, la madurez del equipo y los objetivos planteados.

11. Transferencia de diseño de interfaces de usuario a código. Revisión sistemática

La transferencia de diseño de interfaces de usuario (UI) a código constituye un proceso fundamental en el desarrollo web contemporáneo, dado que impacta directamente en la eficiencia, la calidad del software y la experiencia del usuario. Este estudio, basado en una revisión sistemática de la literatura, identifica y analiza los principales métodos, técnicas y herramientas empleadas en dicha transferencia. Entre ellas se destacan dos enfoques principales: la conversión de mockups a código mediante herramientas como Figma o Sketch, y la detección de elementos de interfaz utilizando algoritmos y redes neuronales. Los resultados muestran que los métodos basados en mockups permiten una representación visual clara del diseño final, reduciendo errores en la implementación y optimizando la calidad del código. Por otro lado, los enfoques basados en detección automática dependen de la precisión de los algoritmos y de la calidad de los insumos utilizados.

Asimismo, el estudio resalta la importancia de la comunicación entre diseñadores y desarrolladores, ya que una coordinación efectiva asegura la correcta traducción de la visión creativa en código funcional.

Transferencia de diseño UI a código



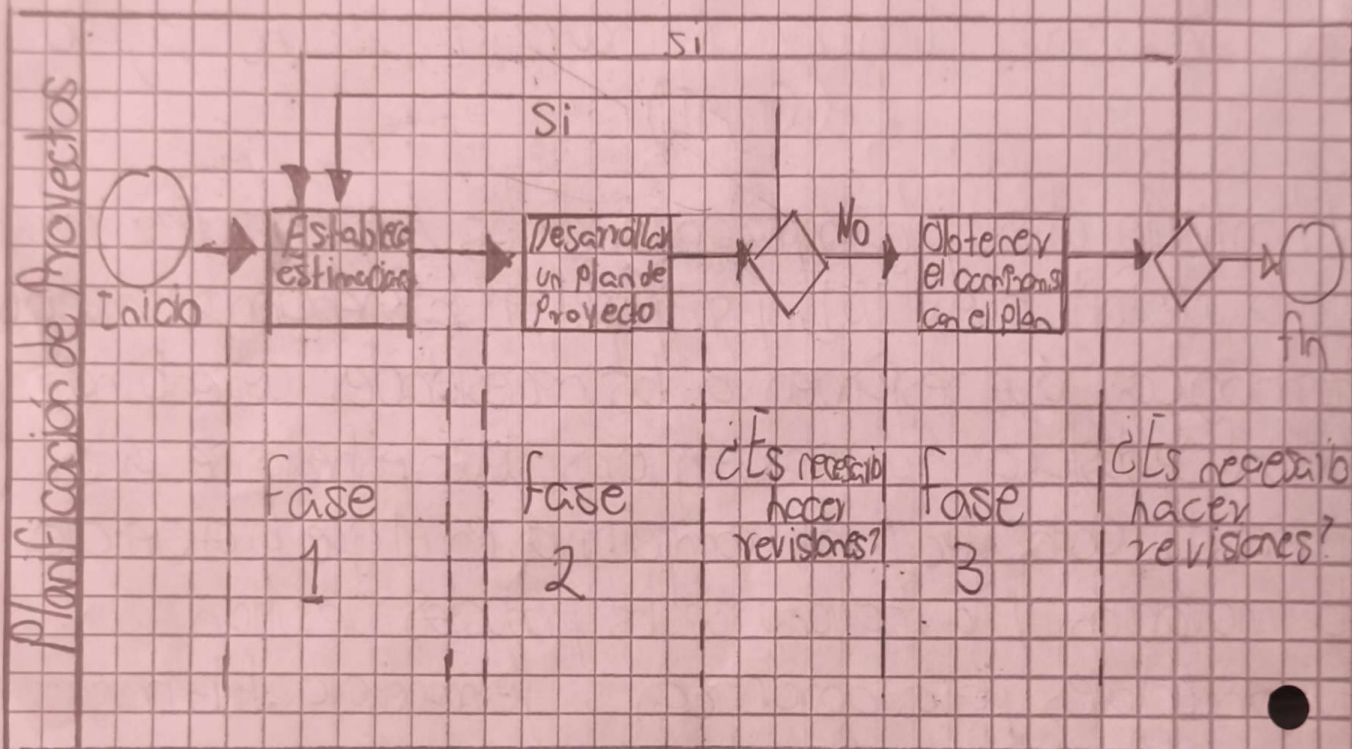
Reflexión: La investigación pone de relieve que la transferencia de interfaces de usuario a código es mucho más que un proceso técnico: es un punto de encuentro entre la creatividad del diseño y la precisión del desarrollo. Reflexionar sobre los enfoques analizados nos permite entender que, aunque las herramientas de mockups y los algoritmos de detección automática ofrecen ventajas significativas en eficiencia y reducción de errores, el verdadero valor se alcanza cuando existe una comunicación fluida entre diseñadores y programadores. Esta sinergia asegura que la visión estética se transforme en funcionalidad real sin perder la esencia de la experiencia de usuario.

15. Mejoras al proceso de planificación de proyectos de software usando el modelo de madurez de proyecto integrado (CMMI)

El artículo aborda la importancia de las Empresas Desarrolladoras de Software (EDS) y su papel en la creación de soluciones tecnológicas que responden a las necesidades del entorno actual. Se destaca que estas empresas deben enfrentarse a un mercado cada vez más competitivo, en el que la eficiencia, la innovación y la calidad de los productos resultan fundamentales para garantizar su permanencia. Además, se menciona que, a medida que el sector avanza, las organizaciones requieren metodologías ágiles y herramientas tecnológicas que permitan optimizar los procesos de desarrollo, reducir tiempos y minimizar errores.

En este contexto, las EDS no solo deben centrarse en la parte técnica de la construcción de software, sino también en la gestión adecuada de proyectos, la comunicación con los clientes y la adaptación a cambios constantes. El texto resalta la necesidad de integrar modelos organizativos flexibles que potencien la colaboración entre equipos y promuevan la formación continua del talento humano.

gráfica:



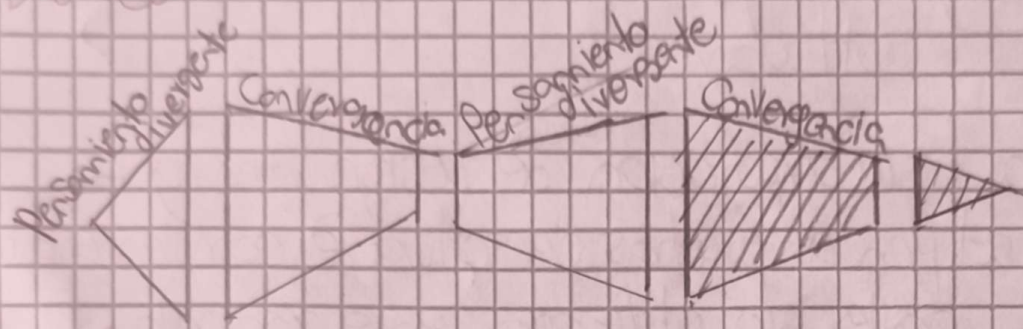
Reflexión: La lectura permite reflexionar sobre el papel estratégico que cumplen las empresas desarrolladoras de software en la sociedad actual donde la tecnología es motor de transformación y competitividad. El texto evidencia que estas organizaciones no pueden limitarse únicamente a producir soluciones técnicas, sino que deben integrar metodologías ágiles, comunicación efectiva y gestión de proyectos para garantizar resultados sostenibles. Esto nos invita a reconocer que la innovación en el sector depende tanto del dominio técnico como de la capacidad de adaptación y aprendizaje continuo de los equipos. Asimismo, el énfasis en la colaboración y la flexibilidad organizativa resalta que el capital humano es el verdadero valor diferenciador en un mercado dinámico.

6. Design Thinking en la planificación de pruebas de Software

El artículo aborda la incorporación de Design Thinking en la planificación del plan de pruebas de software, complementando las metodologías ágiles ya utilizadas en el desarrollo. Señala que, debido a la intangibilidad del Software y la dificultad para comprender los requerimientos del usuario, es fundamental que los encargados de las pruebas se enfoquen en entender lo que el usuario realmente necesita. Para ello, Design Thinking aporta pilares como empatía, colaboración y creación de prototipos, lo que favorece una mejor interpretación de los requerimientos y una actitud más asertiva por parte del equipo de pruebas.

En las conclusiones se destaca que el uso de Design Thinking mejora los resultados respecto a las metodologías tradicionales en la planificación de pruebas, ofreciendo un enfoque más dinámico y adaptable. Aunque aún es una práctica relativamente nueva en comparación con métodos de desarrollo anteriores.

Modelo:



Empatía



Definición



Ideación



Prototipado



Testeo

Re-reflexión: La propuesta de integrar Design Thinking en la planificación de pruebas de software nos invita a reflexionar sobre la importancia de poner al usuario en el centro del proceso de desarrollo. Tradicionalmente las pruebas se han concebido como una etapa técnica enfocada en detectar fallos, pero este enfoque introduce una perspectiva más humana, basada en la empatía y la comprensión de necesidades reales. Al incluir la colaboración y la creación de prototipos, se transforma la forma en que los equipos de prueba se relacionan con el producto y con los usuarios, generando mayor valor y reduciendo los brechos entre expectativas y resultados.

Nota:

Curso:

Institución:

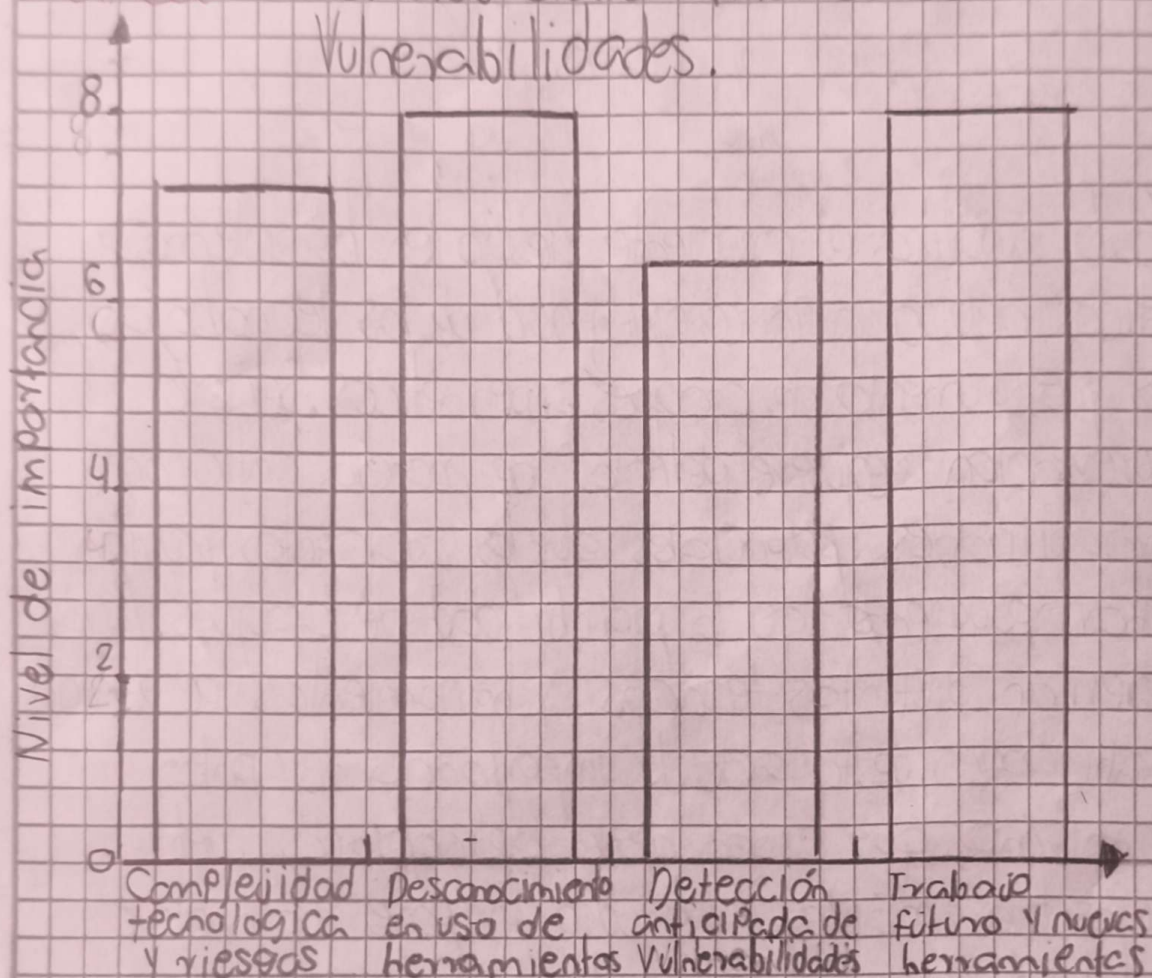
Materia:

Profesor:

17. Guía de ataques, vulnerabilidades, técnicas y herramientas para aplicaciones web

El artículo analiza el creciente riesgo en los sistemas informáticos debido a la complejidad de las tecnologías actuales y el aumento de ataques cibernéticos. Señala que la prevención requiere detectar de manera anticipada las vulnerabilidades potenciales que los atacantes podrían aprovechar, reduciendo así la probabilidad de ataques. Para ello, se revisan distintas técnicas y herramientas de detección de vulnerabilidades, destacando la importancia de contar con mecanismos que cubran desde aplicaciones web hasta dispositivos móviles. El estudio propone una matriz de trazabilidad entre ataques, vulnerabilidades, técnicas y herramientas, que sirve como guía para que las organizaciones identifiquen qué herramientas aplicar frente a vulnerabilidades concretas, facilitando la prevención de ataques.

Gráfica: Aspectos clave en la detección de Vulnerabilidades.

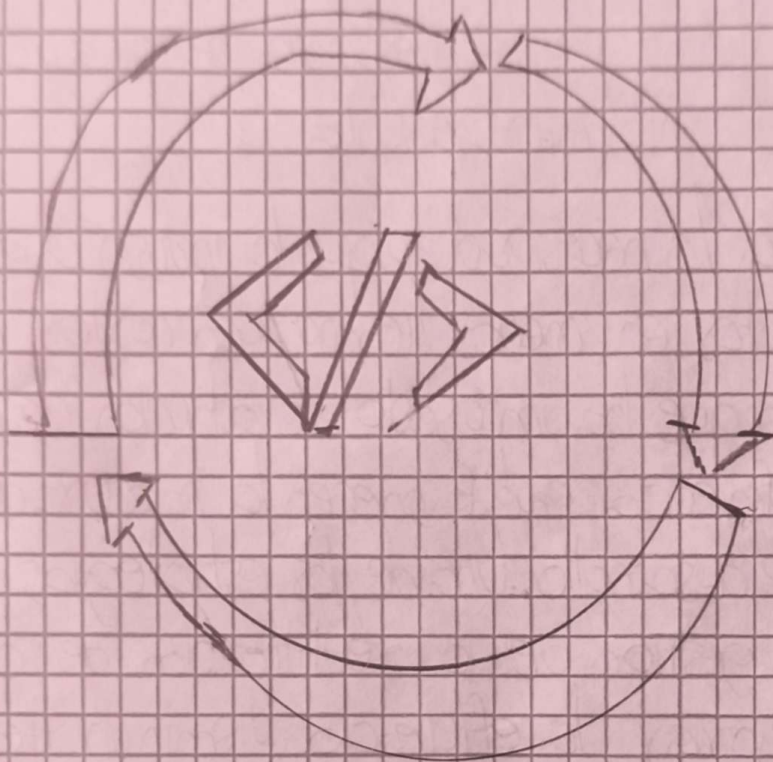


Reflexión: El artículo resalta un aspecto crítico en la actualidad, la seguridad informática como pilar esencial frente a la creciente complejidad tecnológica y el aumento de ciberataques. La idea de anticiparse a las vulnerabilidades demuestra que la prevención es más valiosa que la corrección tardía, ya que no solo evita pérdidas técnicas, sino también impactos económicos y sociales. La propuesta de una matriz de trazabilidad resulta especialmente significativa, pues brinda a las organizaciones una guía clara para seleccionar herramientas según el tipo de vulnerabilidad lo cual fomenta un enfoque estratégico y ordenado de la seguridad.

10 La gestión de reutilización de software en el Modelo de la Calidad para el Desarrollo de aplicaciones Informáticas en Cuba

El artículo aborda la necesidad de que la industria de software desarrolle productos en menos tiempo, con costos reducidos y alta calidad, lo que ha impulsado la adopción de normas y modelos de referencia que favorecen la maduración de los procesos de desarrollo. Una de las estrategias más relevantes es la gestión de la reutilización de software, que permite mejorar la eficiencia, disminuir tiempos de desarrollo y elevar la calidad del producto final. Sin embargo en el contexto cubano la implementación de estos modelos resulta costosa. Como alternativa, la investigación propone un conjunto de buenas prácticas de gestión de reutilización, basado en una revisión bibliográfica y sintetizado en los procesos de Desarrollo de la solución Técnica (DST) y Gestión de procesos de la Organización (GPO).

Gráfica:

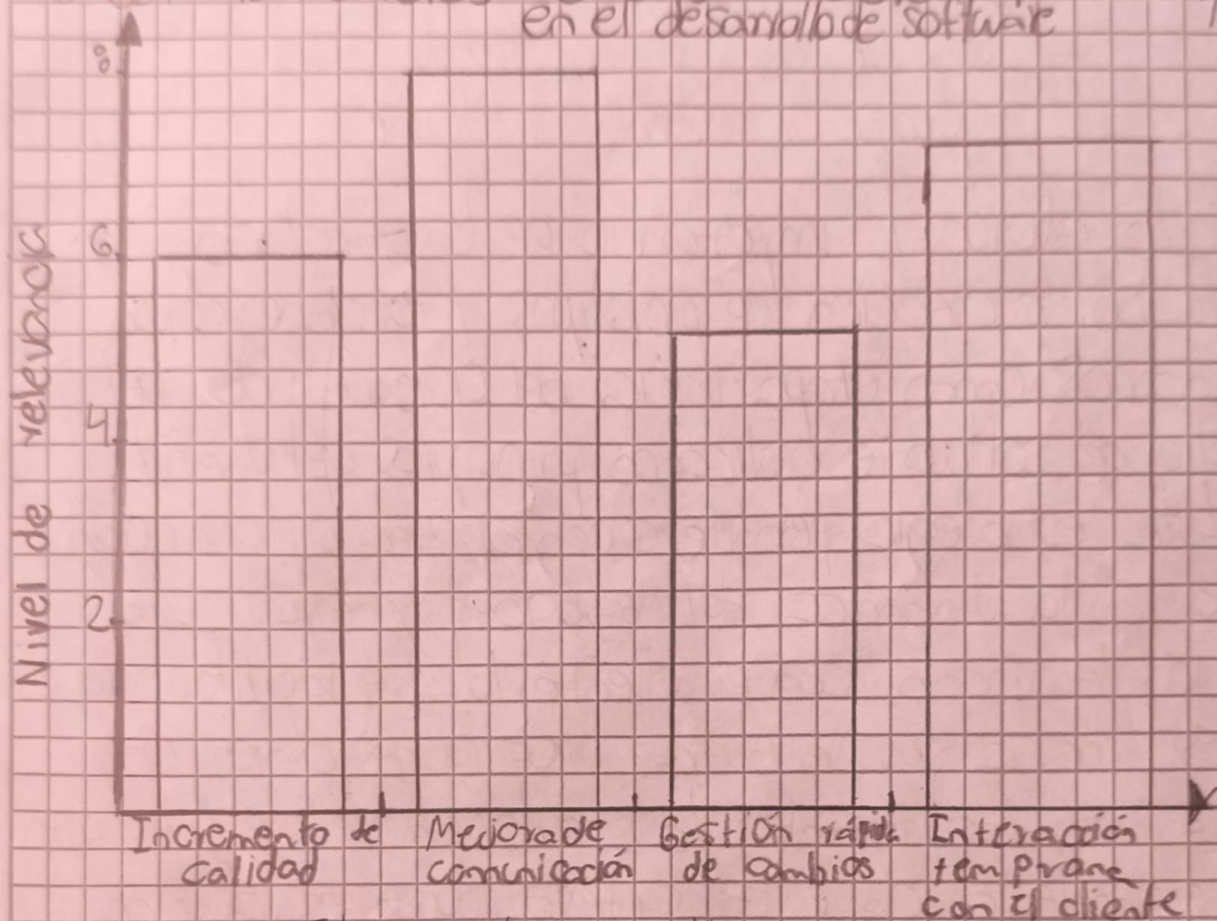


Reflexión: El artículo pone de relieve un tema clave para la industria del software: la necesidad de crear productos de calidad en menos tiempo y con menor costo, sin sacrificar eficiencia ni confiabilidad. En este sentido, la reutilización de software se presenta como una estrategia fundamental, ya que no solo agiliza los procesos, sino que también permite aprovechar conocimientos y componentes previos, reduciendo errores y asegurando resultados más consistentes. Sin embargo, el caso cubano refleja las dificultades de aplicar modelos internacionales debido a sus altos costos y la falta de recursos. La propuesta del MCDAI, con prácticas adaptadas a la realidad nacional, demuestra que es posible lograr estándares de calidad sin metodologías extranjeras.

19. Implementación de métodos ágiles para la simulación de casos de uso y prototipado en el proceso de desarrollo de software

El artículo aborda la importancia de la simulación de casos de uso, la creación de prototipos y la generación de escenarios como etapa inicial del ciclo de vida del desarrollo de software, aplicando principios del diseño ágil. Esta estrategia permite alcanzar una aproximación cercana a la dinámica del negocio y generar una interacción temprana con el cliente, lo que facilita orientar el proceso hacia sus verdaderas necesidades antes de entrar en fases más formales. La simulación se presenta como una herramienta clave en contextos donde es la única vía para anticipar los resultados esperados del sistema. Al presentar de manera visual y abstracta las funcionalidades, se logra identificar límites, objetos y relaciones entre actividades o procesos, evitando desviaciones y manteniendo el foco en los objetivos centrales. La participación temprana del cliente aporta beneficios significativos, ya que incrementa la calidad del diseño, agiliza las conexiones y eleva el grado de aceptación de la solución final.

Gráfica: Beneficios de la simulación y Prototipado en el desarrollo de software



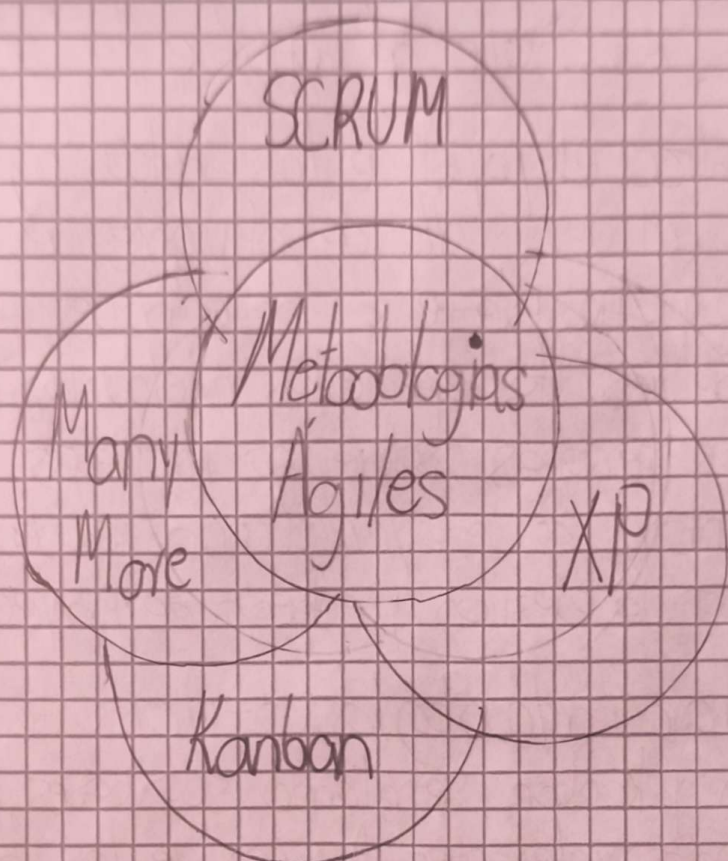
Reflexión: El artículo resalta el valor de la simulación, los prototipos y los escenarios como herramientas esenciales en las etapas iniciales del desarrollo de software. Reflexionar sobre este enfoque permite entender que no se trata solo de representar visualmente un sistema, sino de crear un puente entre el cliente y los desarrolladores desde el inicio del proyecto. La participación temprana del usuario no solo garantiza que el producto se acerque a sus verdaderas necesidades sino que también fortalece la comunicación y reduce riesgos de desviaciones costosas en fases avanzadas. Además, aplicar principios ágiles en esta etapa inicial fomenta la flexibilidad, permitiendo que los cambios sean asumidos como parte natural del proceso.

20. Comparación de metodologías ágiles para el desarrollo de software

El artículo analiza el papel de las metodologías ágiles en la industria del desarrollo de software, destacando su carácter adaptativo frente a los enfoques predictivos tradicionales. Se resalta que estas metodologías priorizan a las personas sobre los procesos y han permitido afrontar con mayor eficacia los desafíos relacionados con la gestión y entrega de proyectos.

El estudio compara metodologías ampliamente utilizadas como **XP**, **Scrum** y **Kanban**, señalando sus bases, principios y diferencias más relevantes. A pesar de las variaciones en su aplicación, todas coinciden en la búsqueda de adaptabilidad, la entrega de productos de calidad y la satisfacción del cliente. Sin embargo, el análisis revela que ninguna de las metodologías estudiadas aborda de manera integral aspectos como la eliminación de desperdicios, la optimización de procesos y la entrega continua de valor. En conclusión, la investigación destaca que cada metodología resulta valiosa en contextos específicos pero la combinación de enfoques ágiles y que constituye para lograr productos de alta calidad y procesos más sostenibles.

Gráfico:



Reflexión: El análisis de las metodologías ágiles muestra cómo estas han transformado la industria del software al priorizar la flexibilidad, la colaboración y la entrega de valor continuo al cliente. Sin embargo, la reflexión más importante radica en reconocer que, aunque marcos como Scrum, XP o Kanban ha demostrado ser efectivos, ninguno logra cubrir de manera integral todos los aspectos que garantizan un desarrollo realmente eficiente y sostenible. La propuesta de integrar principios ágiles evidencia la necesidad de evolucionar constantemente, eliminando actividades innecesarias y optimizando los procesos.

Institución:	Curso:	Nota:
Profesor:	Materia:	
Nombre:	Fecha:	